

SQAT in plain English — the whole thing, made simple

Read this like a friend explaining it. Every term = what it really means + an easy example. Write it in your own words like this in the exam.

CHAPTER 1 — Basics of Software Quality Assurance

What is software? — Not just the code. It's the code + the how-to steps + the manuals + the data together. Two kinds: **ready-made** you buy (like MS Office) and **custom-made** for one company. *Easy example:* Buying Office = ready-made; a shop getting its own billing app built = custom.

What is software quality? — How well the software does what people need, and how happy it makes them. *Example:* A calculator that always gives the right answer and is easy to use = good quality.

The 3 big project problems — **Time** (finishing late), **Cost** (going over budget), **Scope** (missing features or full of bugs).

Two famous disasters — *Therac-25*: a hospital radiation machine; a software bug gave patients too much radiation and killed people. *Ariane 5*: a rocket blew up 37 seconds after launch (1996) because of a software mistake — the most expensive bug ever.

What is testing? — Running the software on purpose to catch mistakes and check it does its job. *Example:* Typing a wrong password to make sure the app blocks you.

Static vs Dynamic checking — *Static* = just **reading** the code/documents without running it (like proofreading an essay). *Dynamic* = actually **running** the software to see what breaks.

SQA (Quality Assurance) — Stopping problems before they happen by setting good rules and a good process. It's about the **process**, it does **not** run the program, and **everyone** on the team does it. *Example:* A rule that every bit of code must be checked by someone else before it's added.

SQC (Quality Control) — Catching problems by actually **testing** the finished product. It **runs** the program, and **only the testing team** does it. *Example:* Running the tests and writing down the bugs. *Remember:* SQA = prevent (before) · SQC = catch by running it (after).

Verification vs Validation — *Verification* = "Did we build it the **right way**?" (did we follow the plan). *Validation* = "Did we build the **right thing**?" (is the user actually happy).

Why quality is hard — Big code, tricky logic, heavy usage, constant changes, and cost all make it hard.

SqE (Quality Engineering) — The whole effort to make quality happen. Three steps: **Plan** → **Do** → **Decide** (pass or fail).

Error, Fault, Failure, Defect (very important — learn this chain): *Error* = a **human mistake**. → *Fault* = the **wrong thing left in the code** by that mistake. → *Failure* = when that wrong code makes the app **actually misbehave**. → *Defect* = the general word for all of these (also called a **"bug"**). *Example:* A programmer forgets to handle dividing by zero (**error**) → a bad line sits in the code (**fault**) → the app crashes when someone divides by zero (**failure**).

Can we test everything? — No. There are too many possible inputs and situations, so testing everything is basically impossible. We test the important stuff.

CHAPTER 2 — Software Quality

Two ways to see software — Users see it from **outside**, like a **black box** (they only see it work). Developers see it from **inside**, like a **clear box** (they see the code).

The 5 views of quality (by Kitchenham & Pfleeger) — five ways people judge quality: 1. **Mystical** — you can't define it, you just *feel* it's good. 2. **User** — good if it does what users want and lots of them are happy. 3. **Manufacturing** — good if it's built exactly to the rules, right the first time (**CMM and ISO 9001** follow this). 4. **Product** — if it's well-built inside, it'll be good outside too. 5. **Value-based** — good if it's worth the money you pay.

Why measure quality? — So you use real numbers, not guesses: to set a starting point, to improve, and to plan ahead.

Quality Factor vs Quality Criterion — *Factor* = the **big outside quality** people notice (reliability, correctness, efficiency). *Criterion* = the **smaller inside thing** that makes it happen (modularity, maintainability). *Remember:* Reliability = **factor**; Modularity = **criterion**. (This is **McCall's** idea.)

Some qualities in plain words — *Security* = keeping out people who shouldn't get in. *Efficiency* = not wasting the computer's power/memory. *Usability* = how easy it is to learn and use. *Reliability* = the chance it keeps running without breaking over time.

ISO-9126 — the most famous quality checklist. It has **6** main qualities: **Functionality** (does what's needed), **Reliability** (keeps working), **Usability** (easy to use), **Efficiency** (doesn't waste resources), **Maintainability** (easy to fix/improve), **Portability** (easy to move to another device/OS). *Important:* **Security is NOT one of the 6.**

Two sides of expectation — Users want it "good enough for the **price**." Makers want it "good enough for the **cost**."

CHAPTER 3 — Maturity Models

Software process — just the set of steps a team follows to build software. A clear process means you can **repeat it, measure it, and improve it**.

Why improve the process? — better **quality**, finish **faster**, spend **less**.

Three models — **CMM** grades your *development* process · **TMM** grades your *testing* process · **TPI** helps *improve* your testing process.

CMM — 5 levels (how grown-up a company's process is): 1. **Initial** — messy, no real process, depends on a few good people. 2. **Repeatable** — basic planning, can repeat past success. 3. **Defined** — has its own proper written process everyone follows. 4. **Managed** — uses **data/numbers** to control the process. 5. **Optimizing** — always improving itself. *Remember:* 4 = **Managed**, 5 = **Optimizing** (don't swap them).

TPI — 4 steps to improve testing — pick what to improve → check where you are now → decide where you want to be → make the change.

TMM — 5 levels (made by **Ilene Burnstein**), how grown-up your testing is: 1. **Initial** — test only after coding, no plan. 2. **Phase Definition** — start planning tests properly. 3. **Integration** — build a real testing team, test throughout. 4. **Management and Measurement** — review and measure quality everywhere. 5. **Optimization** — use data to stop bugs before they happen.

CHAPTER 4 — Software Quality Assurance

3 ways QA deals with bugs: *Prevention* — stop bugs getting in at all. *Example:* dropdowns so users can't type wrong stuff. *Reduction* — find and remove bugs that got in. *Example:* code reviews and testing. *Containment* — if a bug slips through, limit the damage. *Example:* auto-save so you don't lose your work.

2 ways to prevent bugs — remove the **cause** (train people better) or **block the mistake** (don't let someone divide by zero).

Ways to reduce bugs — mainly **Inspection** (carefully reading the code/design) and **Testing**.

2 ways to contain bugs — **fault tolerance** (auto-recover, like undo/rollback, or run several versions = **NVP**) and **safety measures**.

Safety words (easy) — *Safety* = no accidents. *Accident* = a serious harmful failure. *Hazard* = the risky condition that could lead to an accident. *Example:* autopilot stopping a pilot's mistake = safety.

Ways to build software — **Waterfall** (step by step), **Iterative** (in small pieces), **Spiral** (with risk checks), **Agile** (fast, test-first, pair programming).

Key point: prevention is **never 100%** — that's exactly why we also need reduction and containment.

CHAPTER 5 — Software Quality Engineering

SQE — making sure the software hits its quality goals while keeping **cost and risk low**.

3 stages of testing work — *Before (Pre-QA)* = plan the tests — most big decisions happen here. *During (In-QA)* = run the tests, note the bugs. *After (Post-QA)* = measure and improve — and this actually runs **alongside**, not only at the very end.

Test Plan vs Test Case vs Test Suite — *Test Plan* = the **big-picture plan** (what, when, who, how). High-level. *Test Case* = **one small check**: this input should give this result. Low-level. *Example*: "Enter correct username + password → user logs in." *Test Suite* = a **bunch of test cases** grouped and run together.

Regression testing — re-running your **old tests** after a change, to make sure you didn't break anything. You **don't write new tests** for it. *Example*: After adding a "Remember me" box, re-run the old login tests to check login still works.

3 team setups — *Vertical* = one team does **all testing for one product**. *Horizontal* = one team does **one type of test across many products**. *Mixed* = both.

CHAPTER 6 — Software Testing

Testing (simple) — run the software, watch what happens; if it breaks, find and fix the cause; if it doesn't, you trust it a bit more.

The 7 Principles (each with an easy example): 1. **Testing finds bugs but can never prove there are none left.** *Example:* passing all tests doesn't mean zero bugs. 2. **You can't test everything** — too many possibilities, so focus on what matters. 3. **Test early** — bugs caught early are much cheaper to fix. 4. **Bugs cluster** — most bugs hide in a few parts of the app. 5. **Pesticide paradox** — running the same tests over and over stops finding new bugs; change your tests. 6. **It depends on context** — a banking app is tested harder than a game. 7. **No-bugs-but-useless** — a bug-free app is still a failure if nobody wants to use it.

4 levels of testing + who does it: *Unit* — **developer** tests one small piece. *Example:* testing one function. *Integration* — **tester** checks pieces working together. *System* — **tester** checks the whole app. *Acceptance* — the **customer** checks it's what they wanted.

Developer vs Tester — the **developer builds** it and wants it to work (positive mindset). The **tester tries to break** it and wants to find bugs (critical mindset).

White-box vs Black-box vs Gray-box — *White-box* = you **can see the code** and test the inside (done by **developers**). *Example:* checking every if/else path. *Black-box* = you **can't see the code**, you just test what it does from outside (done by **testers**). *Example:* type a password, see if login works. *Gray-box* = a **mix** of both.

When to stop testing? — NOT just "we stopped finding bugs." Better: stop when you **hit your quality goals** (stopping only because time/money ran out is a bad reason).

Manual vs Automated testing — *Manual* = a person tests by hand. Thorough but slow and tiring. *Automated* = a tool runs the tests. Fast and repeatable. *Key point:* automation **can't do everything** — things like "is this easy to use?" still need a human. **Automation can't fully replace manual.**

What to automate / not — Automate the boring **repeated** tests (like regression). Don't automate **usability**, one-time, or gut-feeling tests.